

创始人手册 构建 AI 原生初创公司

Anthropic 出品 · 2026年5月

The Founder's Playbook: Building an AI-Native Startup

目录

第一章 创业生命周期，2026版全新解构

第二章 创始人的角色正在改变

第三章 创意阶段 (Idea Stage)

第四章 MVP 阶段

第五章 发布阶段 (Launch Stage)

第六章 规模化阶段 (Scale Stage)

第七章 同一份工作，新的游戏规则

资源 参考链接与扩展阅读

创业生命周期，2026版全新解构

AI 正在重塑初创公司的构建方式。从未写过一行代码的创始人，如今已能上线生产级应用；精简10人独角兽也从励志故事变成了刻意为之的战略规划。

2026年，AI

能够编写生产代码、开展市场调研、梳理竞争格局、起草融资材料，并自动化运营 workflow。AI 消除了即使是有经验的技术型创始人在整合工具时也曾面临的陡峭学习曲线，从根本上拉平了谁能创业、谁能做产品的起跑线。好想法在今天比任何时候都更能让创始人走得更远。智能体编程将过去需要一支工程师团队才能完成的工作，压缩成一位创始人就能独立交付的成果。

传统创业增长弧线的预设路径是：验证 → 融资 → 招人 → 构建 → 再融资 → 增长 → 继续招人 → 循环往复。现在，AI 打破了每进入一个新阶段就需要更大团队、不同技能集和新一轮融资的既定预期。

本手册依据这些新现实，重新绘制了创业旅程的四个核心阶段地图：

阶段	核心问题	AI 的主要角色
创意阶段 (Idea)	这是否值得构建？	研究伙伴、验证助手
MVP 阶段	如何快速构建并找到 PMF？	工程团队、架构顾问
发布阶段 (Launch)	如何让增长可复制？	运营系统、合规工具
规模化阶段 (Scale)	如何建立可防御的护城河？	GTM 引擎、知识底座

如果你已准备好绘制从想法到退出的最短路径，请继续读下去。

创始人的角色正在改变

创始人的定义过去来源于他们能做什么：技术型创始人写代码，非技术型创始人负责商业运营和拿下交易。但2026年可用的模型、系统和 AI 智能体，已经溶解了能构建的人与有值得构建的想法的人之间的壁垒。

AI 原生初创公司正从根本上改变创始人的含义。如今没有工程背景的人也能构建生产级软件，而技术背景深厚但商业经验有限的创始人，也能轻松产出市场进入策略、财务模型和高度精良的 pitch deck。

历史上，创始人大部分时间处于执行模式：写代码、管理人员、处理日常运营。在 AI 原生创业公司里，创始人角色大幅减少了个人执行者的成分，转而更多地成为智能体协调者——指挥那些能读取文件、运行命令、执行代码甚至浏览网页的专业 AI 助手。创始人的注意力向上迁移：产生创意，并指挥承载这些创意的系统（AI 智能体、工具及精简团队）。

AI 作为核心基础设施最具革命性的意义，是解放了拥有领域专长的非技术型创始人。当创始人群体扩展到工程背景之外，你会看到由各种不同生活经历的人构建的初创公司，解决传统技术创始人路径从未优先考虑的真实问题。

AI 工具能力：三大核心领域

传统创业模式假设你需要雇工程师构建、雇销售来卖、雇运营来经营。2026年的早期创业公司截然不同——精简是刻意为之的设计，通常只有创始人独自一人或极小的团队。通过将技术和组织发展都以 AI 为基础设施，他们能在扩大团队规模之前，就实现产品验证、早期营收甚至盈利。

1. 对话智能与研究

设想：每个领域都有一位随时待命的顾问。创始人第一年需要了解的一切（如何设置薪资、如何规划产品迭代、如何起草投资人备忘录），过去这些问题的答案都是同一个：找一个懂行的人。现在 Claude 可以提供深度研究（竞争分析、市场规模、财务建模）、文件起草（pitch deck、投资人备忘录、PRD）、战略思维（魔鬼代言人分析、事前分析、情景规划）。

2. 智能体编程 (Agentic Coding)

设想：永远在线、永不被阻塞的工程师。构建软件曾需要技术型联合创始人或外包开发团队。智能体编程工具现在让每一位创始人都能用自然语言描述想要构建的内容，并指挥 AI 以完整工程团队的速度和规模来生成、测试、调试和重构生产级代码库。从「我有一个想法」到「我有一个产品」的时间线已大幅压缩。创始人的角色聚焦于构建什么和为什么，而 AI 负责实际建造。

3. 工作流自动化

设想：按需调用的自动化运营团队。日程安排、更新 CRM、生成周报、维护文档……这些都必须发生，但消耗创始人时间和精力。工作流自动化将这些重复性任务配置成自动执行——CRM 在交易推进时自动更新，周报自动生成，产品文档与产品变更同步更新。Claude Cowork 与你的项目管理工具、沟通工具栈、数据源无缝集成，无需构建和维护这些集成。

创意阶段 (Idea Stage)

每一位创业创始人都从同一个起点出发：一个挥之不去的问题。这是创意碰撞现实的阶段：2026年的创业成功需要一种自律——在证据充分之前，不要开始构建。本阶段的工作是研究、客户发现、竞争分析，以及对不支持假设的证据进行诚实评估，所有这些都要在让 Claude Code 生成第一行生产代码之前完成。

创意阶段目标

创意阶段的核心目标是以研究为导向的验证：在承诺资源开始构建之前，积累充分证据证明真实问题存在，以及你的解决方案能有效解决它。需要依次回答以下几个问题：

- 这个问题是否真实、具体，且足够频繁，值得围绕它构建产品？
- 谁在经历这个问题，那是一个市场吗？
- 是否已有人在解决这个问题？如果有，他们怎么做，效果如何？
- 解决方案需要具备哪些功能才能真正解决这个问题，而我的想法是否满足这些要求？

这些问题的答案汇聚成一个终极问题：这是否值得构建？「人们在费用报销上有困难」是一个观察。「中型企业的财务经理每周花4小时以上对账，因为现有工具无法与会计软件集成」，才是可验证的假设。

创意阶段退出标准

创意阶段的退出条件是找到问题-解决方案契合 (Problem-Solution Fit)。当你能对以下三个问题全部回答「是」时，就可以离开创意阶段：

- 问题是否真实且具体？
你能清楚说出谁在经历这个问题、多频繁遇到、严重程度如何、他们目前如何应对。
- 你的解决方案是否针对真实问题？
不是你最初假设的问题，而是验证过程揭示的问题。有时两者一致，但并非总是如此。
- 你是否有足够的信号证明值得构建？你需要足够的定性证据，使进入 MVP 阶段成为理性判断而非信仰行为。

创意阶段的核心挑战

挑战一：把构建误当成验证

当技术障碍被消除，充满激情的创始人面临的风险是：跳过创业旅程中最重要的工作——验证想法是否真的是人们需要并会使用的解决方案。即便在当前时代之前，42% 的初创公司因构建了没有人想要的产品而失败。现在，智能体编程大幅压缩了从想法到产品的距离，这一失败率只会

攀升。一个可运行的原型很容易被误认为是在解决真实问题的证据，但并非如此。原型只是用户访谈中的辅助工具——对话本身才是真正的证据。

挑战二：过早扩张

当构建毫不费力且近乎免费时，你可以在商业需求真正验证之前，把执行规模推得很远。智能体编程助手会以同样的热情，在一个根本有缺陷的前提上生成和重构代码库。本阶段的核心指令是：让你的认知始终领先于你的构建。

挑战三：失去客观性

如果你让 AI 寻找支持你已有信念的证据，它就会找到。确认偏误现在配备了一个研究引擎。解决方案是主动要求 Claude 扮演魔鬼代言人，专门寻找反对你想法的最强反驳论点。这是贯穿每个创业阶段的核心实践。

Claude 如何帮助创意阶段的创始人

选择正确的 Claude 界面

如果任务是……	使用界面	原因
一个问题、一次改写、快速头脑风暴	Chat	快速、对话式、无需配置
研究、分析，或基于文件/系统生成完整文档	Claude Cowork	文件夹访问、连接器、技能、定时运行
编写、测试或交付软件	Claude Code	代码库访问、差异对比、Git、开发环境

关键使用方式

- 压力测试假设：让 Claude 主动寻找反驳你想法的最强证据；将问题陈述打磨成可测试的假设格式。
- 竞争格局图谱：按层级（直接竞品、间接竞品、潜在收购方、相邻玩家）进行映射，让 Claude 论证每个层级的真实威胁。
- 市场调研：让 Claude Cowork 合成竞品用户评价，挖掘未被满足的需求；构建 TAM/SAM/SOM 模型并压力测试假设。
- 趋势分析：追踪用户讨论你的问题时用的语言；识别类比市场中成功和失败的规律；关注可能加速或威胁机会的趋势。
- 客户发现规划：定义精准目标画像；构建访谈框架，聚焦「上次遇到这个问题时」而非「你会不会使用」；让 Claude 标记引导性或宽泛的问题。
- 访谈后分析：将每次访谈笔记输入 Claude 进行逐条分析；收集5次访谈后，用 Claude Cowork 提炼主题并生成支持假设与挑战假设两份清单。

- 外联自动化：Claude Cowork 可构建联系人列表、起草个性化邮件、通过 MCP 管理日程安排。
- 设计解决方案概念：让 Claude 从各个角度挑战你的解决方案设计：有哪些缺口？存在哪些替代方案？规模化时需要什么前提？
- 用 Claude Code 构建轻量级原型：只构建你的解决方案所依赖的那一个核心交互，放在5位真实目标用户面前测试。

MVP 阶段

许多创始人把 MVP 阶段当作纯粹的构建阶段，但 MVP 阶段本质上仍然是一个证据收集练习。区别在于：你现在收集的是关于解决方案的证据——某个真实可辨的用户群体是否认为产品有足够价值，愿意使用它、回归它、为之付费，并向他人推荐。

MVP 阶段目标

- 将已验证的问题转化为真实用户会真正使用的工作产品：不是功能齐全的完整版本，而是能让真实用户真实使用，并产生 PMF 真实证据的最小、最聚焦的版本。
- 快速移动，同时不积累复利技术债：AI 技术债是指数级的——没有文档记录的架构决策，每次会话都会重新推导，代码库会漂移。
- 从第一天就投资于持久化上下文：跳过规格文档、架构决策和上下文文件（如 CLAUDE.md）的创始人，会撞上一堵可预见的墙——每次新会话都需要重新解释代码库，AI 生成的变更偏离原始愿景。

MVP 阶段退出标准

MVP 阶段的退出条件是产品-市场契合（PMF）的真实证据：特定可辨识的用户群体已发现产品足够有价值，愿意回归（留存）、为之付费（收入）、或向他人推荐（转介绍）。

MVP 阶段的核心挑战

挑战一：智能体技术债

因为 AI 基本上消除了曾经控制哪些内容进入生产的所有自然瓶颈，速度是有保障的。但当速度是 MVP 构建中考虑的唯一变量时，创始人面临积累难以偿还的技术债的风险。AI 技术债是复利的——没有规格文档和架构约束，每次会话都从头重新推导基础决策，这些决策会漂移。最终你得到一个没有内在一致性的代码库——不是因为某个单独的部分很差，而是因为这些部分从来没有被设计成配合工作的。

挑战二：虚假的产品-市场契合

早期动力是创始人可能拥有的最具心理效力的体验之一。但早期牵引力不等于产品-市场契合。发布带来的热情来自于短暂的力量——创始人的朋友、投资人的其他被投公司、一波 Hacker News 流量。遗憾的是，这些都无法可靠地预测第六周或第十二周当初始热情消退后会发生什么。

挑战三：零摩擦的功能蔓延

当构建感觉毫不费力时，总有那么一个更酷的功能值得添加，或者一个边缘情况值得处理。传统的制约因素——真实的工程时间成本——不再以同样的方式存在，添加一个功能需要一个下午而不是一个 Sprint。解决方案是在开始构建之前创建一份书面的范围定义文档，描述产品做什么、刻意不做什么，以及需要什么样的真实用户证据才能证明添加新功能是合理的。

挑战四：因经验不足而带来的安全隐患

智能体编程工具生成的是能运行的代码，而不是天然安全的代码。安全漏洞是隐形的，直到被利用时才会显现——没有自然的反馈回路来提醒首次创始人有什么问题。向真实用户发布一个正式的 MVP 意味着真实的数据、真实的暴露，以及出错时的真实后果。

Claude 如何帮助 MVP 阶段的创始人

步骤一：在开始构建之前定义架构

在打开 Claude Code 之前，先打开 Claude 描述你正在构建的内容：它解决的核心问题、服务的用户，以及未来六个月内实际预期的规模。让它帮你定义应该管理 MVP 构建的架构原则、鉴于约束条件要避免的依赖关系，以及你在此阶段有意识地接受的权衡。

然后将这份输出保存为 `CLAUDE.md` 文件——这是你构建过程的第一个产物，也是每次后续会话所依赖的产物。`CLAUDE.md` 文件作为 Claude Code 的项目级指令，在 Agent SDK 运行时自动读取，功能上是你项目的持久化记忆。

步骤二：定义并执行 MVP 范围

就像你定义并记录了产品的应用架构一样，你也需要在构建第一个功能之前定义 MVP 的范围。Claude 可以帮你创建一份范围文档，描述你的 MVP 产品做什么、刻意不做什么，以及功能修订标准：什么样的来自真实用户的具体证据才能证明在此时添加新内容是合理的。当新功能想法浮现时，使用 Claude 来压力测试它究竟是来自用户的真实信号，还是伪装成产品思考的创始人热情。

步骤三：用 Claude Code 构建 MVP

一旦架构和范围定义完毕，Claude Code 就成为主要的 MVP 构建工具。用它来生成、测试、调试和迭代你的代码库，但将每次会话视为已做决策的执行，而不是加入新决策的机会。每次 Claude Code 会话开始时，重新审视你的范围文档，并提供 `CLAUDE.md` 架构上下文文档。每次会话结束时，用该会话涌现的决策来更新它。目标是一个你能解释其结构的代码库，而不仅仅是一个能运行的代码库。

步骤四：任何用户接触代码之前进行安全审查

Claude 可以做一个有用的初步安全审查，涵盖：身份验证/会话管理、API 响应中的数据暴露、输入验证，以及已知存在漏洞的依赖项。Claude Code Security（发布时处于有限测试阶段）会扫描漏洞。但注意：任何涉及身份验证、密钥或数据处理的内容都需要人

工审查，不可省略。

步骤五：发布前构建度量框架

错误识别早期牵引力为 PMF 的创始人，通常是那些在发布后才开始追踪数据的人。解决方案是在第一个用户到来之前建立你的度量框架：在发布 MVP 之前设置你的留存基准、激活标准，以及第7天和第30天目标。然后定义什么是你的特定产品的假阳性：没有激活的注册、没有留存收入，或者没有重复使用的初始热情。当数据到达时，让 Claude 对你自己的牵引力数字提出对抗性论点：怀疑者会怎么看这些数字？

PMF 判断工具

- Sean Ellis 测试：问你的活跃用户「如果你不能再使用这个产品，你会有什么感觉？」如果超过40%的人回答「非常失望」，这是一个有意义的 PMF 指标。
- 努力测试：产品市场契合前，留存需要创始人持续干预。产品市场契合后，产品开始自己拉动用户。
- 迭代诊断（3次以上迭代仍无进展时）：用 Claude 诊断——这份数据中有没有某个细分群体反应不同？设计价值与体验价值的落差是定位问题还是产品问题？当前产品要找到真正的 PMF，哪些条件必须成立，而这现实吗？让答案决定你是调整、转型，还是回到创意阶段。

发布阶段 (Launch Stage)

如果 MVP 阶段是证明产品值得存在，发布阶段是证明公司值得增长。在发布阶段，创始人必须将早期牵引力转化为可复制的、可持续的增长引擎，同时让产品达到生产就绪状态，并在产品周围构建真正的公司。

发布阶段退出标准

- 增长是可复制的、由渠道驱动的：你不只是在留存用户，你在通过具体渠道可预测地获客，CAC、LTV 和回本周期是你知道并能说清楚的数字。
- 产品能承载生产负荷：基础设施已强化，安全合规到位，在真实生产条件下（而非你测试的条件）稳定运行。
- 运营不再依赖创始人亲力亲为：流程和自动化已到位。你已经不需要亲自处理客服、故障排查、Sprint 规划或汇报。

发布阶段的核心挑战

挑战一：技术债到期了

MVP 阶段为了速度积累的技术债，现在开始计息。生产流量、新功能、复杂度提升，开始暴露当初走的捷径。解决方案：系统性的架构审计找出结构弱点 →
有针对性的重构解决最严重的问题 →
大幅扩展测试覆盖率，使下一轮功能开发不会重新引入同样的问题。

挑战二：创始人成为瓶颈

MVP 阶段创始人事事亲力亲为是优势，到了发布阶段就变成了约束。从「做工作」到「设计做工作的系统」的转变，是创业生命周期中最难的转变之一。典型症状：一个本该一小时搞定的决策拖一周才能轮到你处理；客服请求堆积因为只有你知道答案；很多运营任务只在你本人想起来时才会发生。

解决方案：全面盘点你个人处理的所有事项，识别哪些可以系统化、哪些可以授权、哪些真正需要创始人本人。

挑战三：安全和合规不再可以推迟

有了真实用户、真实数据，以及潜在的企业合同，安全漏洞不再是理论风险，而是真实的暴露风险。合规要求从假设变成现实——一旦你在处理客户数据、支付，或进入受监管行业，一切都适用了。解决方案：在生产规模到来之前（而非之后）进行系统性的安全与合规审查，把每一个发现视为必须修复，而非建议参考。

挑战四：还没准备好就扩张

早期牵引力是真实的，但它是特定于你的早期用户的。过早扩张到显著不同的市场，会引入新的用户行为、合规要求、支付基础设施和基线期望——你的产品并没有围绕这些设计。突然之间，变量太多，你失去了清晰解读自己数据的能力。你还面临在追逐新的、未经验证的受众时忽视原有用户群的风险。

Claude 如何帮助发布阶段的创始人

- 修复技术债：用 Claude Code 做全架构审计，识别脆弱之处和测试覆盖盲区 → 把审计结果喂给 Claude，让它帮你排序修复工作（哪些要在下个版本前修、哪些可以等、哪些是可接受的债）。把 MVP 阶段存在脑子里的架构决策记录进 CLAUDE.md，确保每次后续会话都从共同理解出发。
- 建立替代创始人注意力的系统：用 Claude Cowork 盘点你现在亲自做的所有事，分类为「可完全自动化、需要人但不一定是你、确实需要创始人判断」，然后设计自动化工作流。
- 把安全合规纳入产品工作流：用 Claude Code 扫描 SOC 2/GDPR/HIPAA 等标准的代码级问题，把发现喂给 Claude 优先级排序，设计企业买家需要看到的审计日志和访问控制。AI 扫描是辅助，不能替代专业合规审查。
- 建立产品管理流程：用 Claude 设计 Sprint 节奏、Spec 模板、Bug 分类决策树和周度指标简报；用 Claude Cowork 执行调度、路由、汇报等运营层，让这些在不需要你触发的情况下按时发生。

规模化阶段 (Scale Stage)

在规模化阶段，创始人的角色从构建者重新转向对外的执行官。产品仍然核心，但你的日常工作越来越多是关于公司本身——分析师简报、IPO 路演……同时还要努力维持精简的、以 AI 为中心的结构优势。

规模化阶段目标

规模化阶段的目标是通过积累深度建立可防御的护城河，源自三个维度：

- 产品里编码的专业知识：别人难以复制的领域认知和行业 edge case 逻辑
- 与用户依赖的其他工具的深度集成：用户换掉你的产品，相当于重建整个工作流
- 专有系统数据和工作流：数据飞轮——竞争对手无法重建你的用户行为指纹

从数千用户到数百万，从一个市场到多个市场。此阶段还面临更大的外部压力：公开投资者、分析师、监管机构、企业采购团队、潜在收购方——他们不只评估产品能力，还会审查治理、合规、财务控制和战略叙事。

规模化阶段退出标准

规模化阶段的退出条件不是单个里程碑，而是一个门槛事件：即使创始人不再直接运营日常业务，公司也能可持续运营。通常有三种形式：

- 可持续的规模盈利：不再需要外部资本
- IPO 就绪：增长系统化可审计，产品护城河经得起外部审视，组织运营成熟可持续
- 被收购：同样需要满足以上条件

规模化阶段的核心挑战

挑战一：委托运营层

规模化阶段的运营系统必须在没有人看着的情况下可靠运行。这既是结构挑战，也是心理挑战。核心难点是：识别哪些机构知识只存在于创始人脑子里或未文档化的工作流中，然后把它编码进可文档化、可审计、可转移的系统。手放太快（尤其是交给 AI 自动化系统），关键决策会在缺少创始人背景知识的情况下被做出；手放太慢，你自己就成了瓶颈。

挑战二：规模化技术运营

客户不再只评估你的产品，他们想知道你的组织能否成为可靠的基础设施伙伴。挑战从代码库本身延伸到代码库周围的一切：支持体系、文档、可靠性保障（SLA）。签多年合同的大客户和机构买家，在签约前就要看到这些，签约后还会照着执行。

挑战三：规模化组织职能

财务报告、合规监控、合同管理、客户支持……这些组织基础设施无论有多少人在运营都必须存在。发布阶段是把消耗创始人精力的工作流系统化；规模化阶段则需要构建更广泛、在某些方面更重要的组织职能数组。

挑战四：建立 GTM 职能

有机增长有天花板，大多数规模化阶段的创始人在遇到它之前，从来没有构建过真正的市场进入职能。触顶信号：用户增长曲线趋平、CAC 上升、销售管道只有创始人参与才移动。规模化增长需要建立专门的增长引擎，但大多数创始人从未运营过市场营销、销售、分析师关系项目。

Claude 如何帮助规模化阶段的创始人

把日常任务交给 Claude Cowork

用 Claude 构建「只有你应该做的事清单」——产品叙事决策、董事会关系、企业大单、创始人之间的对话。不在清单上的一切都是授权或自动化的候选。然后检验你已构建的系统是否真正准备好随业务增长扩展：让 Claude 绘制你当前工作流中的瓶颈地图，并模拟你一周不在时每个工作流会发生什么——那些停转的，就是交接标准、升级路径或例外处理仍需打磨的地方。

把技术运营扩展成企业级基础设施

- Claude：起草和维护产品文档、支持 Playbook、SLA
- Claude Code：审计和强化代码库，对抗企业合同要求的可靠性和安全标准；构建日志、监控、事故响应工具、可观测性层，让 SLA 真正可执行
- Claude Cowork：运营企业支持层——工单路由、升级工作流、因产品变更触发的文档更新、续约跟踪、企业客户成功所依赖的汇报节奏
- 三者合力，让小团队拥有大得多的组织的支持能力——这正是签订多年企业合同所需要证明的。

建立真正的 GTM 职能

- Claude：市场细分、消息架构、分析师关系策略、销售 Playbook、面向投资者的指标叙事——每个受众群体都有自己的评估语言，Claude 的工作是将你的产品价值主张翻译成对每个受众段相关的产品营销方法
- Claude Cowork：内容管道、外联序列、分析师简报后勤、PR 节奏、CRM 维护、管道报告——让 GTM 策略转化为真实的商业动作
- Claude Code：交互式 Demo 环境、集成文档、沙箱租户、API 文档——规模化阶段，一段 Loom 视频加上销售幻灯片已不够用；一个构建良好的 Demo 环境能在你开董事会时继续帮你签单

把领域专知转化为 AI 上下文（最重要的竞争壁垒）

通过与 Claude 的深度对话、项目和记忆，把创始人知道的一切——行业术语、监管陷阱、边缘案例、为什么显而易见的答案在这个领域不管用——全部编码进结构化、可检索的上下文。技能可以将重复工作流（如「我如何审计一份商业租约」「我如何对患者入院表格进行分诊」）编码为 Claude 每次都以同样方式执行的可复用程序。随着时间推移，这成为通用 AI 无法匹配的专有知识底座。

通用 AI 医疗账单工具会在 340B 药物项目索赔上出错，但你的工具有专门的逻辑处理它。Claude Code 帮你把行业从业者的共同痛点转化为验证逻辑、提示词优化，或与竞争对手从未听说过的小众行业系统的 MCP 集成。结果：你的产品深度和广度都在持续复利增长，竞争对手根本无法复制。

创造工作流绑定 (Workflow Lock-in)

用户在产品上运行的时间越长，产品就越深地嵌入他们的实际工作方式。他们在上面构建了自动化，训练了团队，接入了数据源和其他工具——切换成本从产品决策变成大规模运营项目。

策略：用 Claude 按集成深度绘制客户地图——对每个客户群体，识别他们在产品上构建了哪些工作流、依赖哪些集成。这展示了产品在哪些地方真正粘住了用户，以及需要往哪些方向走更深。用 Claude Code 快速构建与目标用户依赖的数据管道、项目管理工具等的原生集成，以及让客户不只是使用产品、而是在产品上构建自己产品的 API、Webhook 和 SDK——这是最深形式的绑定。

同一份工作，新的游戏规则

在 AI 时代，创始人的工作没有改变：找到真实问题，构建解决它的东西，把它扩展成一家有价值的公司。改变的是路径。

创业任务	过去	现在（AI 时代）
验证假设	需要数月的用户访谈和调研	一个下午
构建原型	需要技术型联合创始人或外包团队	几次专注的智能体编程对话
发布准备	发布前的混乱冲刺	持续的工作流
规模化运营	早期员工沦为救火队员	越来越多交给 AI，人力专注于真正的判断

瓶颈不再是你能构建什么，而是你选择构建什么。

跨越这四个阶段——创意、MVP、发布、规模化——AI 将季度压缩成了周。那些一直在同一个方向上、在一致的基础设施上持续构建的创始人，现在拥有了真正难以复制的东西。

参考链接与扩展阅读

构建相关文档

Building AI Agents for Startups

介绍初创公司如何使用智能体，随着规模化减少对创始人的依赖。

Claude Code 文档

从初始安装到高级智能体工作流的完整指南。推荐从「Claude Code 工作原理」概述开始。

Claude Code 最佳实践

涵盖 Anthropic 内部和工程团队验证的模式——上下文管理、权限、规划和验证工作流。

使用 CLAUDE.md 文件

讲解如何为你的特定代码库配置 Claude Code。MVP 阶段创始人的必读资料。

Claude Code 高级用户技巧

来自 Claude Code 团队自身的工作流模式，包括并行会话和验证循环。

Claude Cowork 入门

介绍团队如何设置 Claude Cowork 并实施技能、插件等功能，扩大其在初创公司的影响力。

教程 (claude.com/resources/tutorials)

可搜索的特定任务动手演练列表。

创始人案例

三家 YC 初创公司如何用 Claude Code 构建公司

HumanLayer (F24)、Ambral (W25) 和 Vulcan Technologies (S25) 如何用 Claude 快速将原型推向市场，并用智能体编程工作流扩展 AI 驱动的平台。

GC AI

创始人利用领域专长构建了一个响应式的、Claude 驱动的法律平台，专为内部法律团队的实际工作方式设计：公司特定剧本、跨职能利益相关方、可变风险容忍度阈值。

Carta Healthcare

使用 Claude 驱动临床数据抽取平台，每年处理22,000例外科手术案例，将数据抽取时间减少66%。

Anything

由 Claude 和 Agent SDK 驱动，已帮助150万用户将想法变成可运行的软件产品，无需编写代码。一位非技术创始人用它构建并正在销售一个完整的招聘平台。

Cogent

应用 AI 实验室，构建智能体自动化企业安全任务（调查、优先级排序、修复），使用 Claude 作为推理层。

Airtree

将 Claude Cowork 作为运营基础设施中心，统一了原来分散在十几个工具和团队中的数据。现在一个人构建一个工作流自动化，组织里所有人都能使用它。

Duvo

构建智能体运行采购、供应链和品类管理流程，跨越 ERP、供应商门户、电子表格、邮件，甚至电话。完全基于 Claude 构建，使用 Agent SDK 跨工作流编排。

Kindora

由一位非营利机构高管构建，用 Claude Sonnet 智能匹配慈善机构与捐助者。过滤数千个候选匹配后，通过 MCP 连接器让非营利机构直接在 Claude 中使用寻访工具。

Wordsmith

由律师转型 CTO 创立，为内部法律团队提供可靠的 AI 驱动法律技术。Claude 是合同审查、协议起草和文件审阅能力的推理引擎，工程团队用 Claude Code 构建和演进平台本身。

创业支持与机会

Anthropic Startups Program

为与 Anthropic VC 合作伙伴合作的初创公司提供免费 API 额度、最高级别的公开速率限制，以及独家创始人活动和研讨会邀请。

Claude 社区

面向构建者的论坛和社区空间。

在线学习资源

会议、网络研讨会、直播和录播内容。